

in the `galleons`, `sickles`, and `knuts` properties: this property isn't based on a backing variable named `_total`. Enter the following into the interactive shell:

```
>>> import wizcoin
>>> purse = wizcoin.WizCoin(2, 5, 10)
>>> purse.total
1141
>>> purse.total = 1000
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: can't set attribute
```

You might not like that your program immediately crashes when you attempt to change a read-only property, but this behavior is preferable to allowing a change to a read-only property. Your program being able to modify a read-only property would certainly cause a bug at some point while the program runs. If this bug happens much later after you modify the read-only property, it would be hard to track down the original cause. Crashing immediately allows you to notice the problem sooner.

Don't confuse read-only properties with constant variables. Constant variables are written in all uppercase and rely on the programmer to not modify them. Their value is supposed to remain constant and unchanging for the duration of a program's run. A read-only property is, as with any attribute, associated with an object. A read-only property cannot be directly set or deleted. But it might evaluate to a changing value. Our `WizCoin` class's `total` property changes as its `galleons`, `sickles`, and `knuts` properties change.

When to Use Properties

As you saw in the previous sections, properties provide more control over how we can use a class's attributes, and they're a Pythonic way to write code. Methods with names like `getSomeAttribute()` or `setSomeAttribute()` signal that you should probably use properties instead.

This isn't to say that *every* instance of a method beginning with *get* or *set* should immediately be replaced with a property. There are situations in which you should use a method, even if its name begins with *get* or *set*. Here are some examples: